AgentWebBench: A Reproducible Benchmark for Browser-Mediated Web Workloads

Anonymous Author(s)

Abstract

Browser-mediated AI systems are a growing source of web traffic, yet no public benchmark captures their behavior at the HTTP request level. Without reproducible traces, measurement studies cannot characterize agentic workloads, and systems researchers cannot evaluate cache policies, CDN placement, or admission schemes under realistic browser-automated request streams.

We present AgentWebBench, a benchmark framework built on three design principles: real browser execution via BrowserUse, a common request-level trace schema, and replay-ready artifact exports. Our public release, `browseruse-live-v2`, contains 100 sessions across 10 task families, totaling 14,570 requests and 338.5 MiB of transferred bytes.

The release exposes rich workload structure. Mean request volume spans a 22.8× range across task families (16–365 requests per run). Scripts and stylesheets dominate transferred bytes (66.5%), while HTML—the ostensible content—accounts for only 3.0%. Replaying traces into six cache policies reveals strong policy sensitivity: GDSF achieves a 58.5% hit rate at 1 MiB while LRU achieves only 23.4%. At 10 MiB, LRU surges to 89.1% while frequency-based policies lag, revealing that the optimal policy depends on cache size—a key finding for CDN operators managing agentic workloads.

AgentWebBench is designed as a release protocol, not a one-time dataset. All traces, collection infrastructure, and analysis pipelines are publicly available, enabling the research community to extend the benchmark with additional agents, tasks, and regions.

CCS Concepts

• **Networks** → **Network measurement**; • **Information systems** → *Web applications*; • **Computer systems organization** → *Cloud computing*.

Keywords

web measurement, browser automation, AI agents, caching, benchmark, reproducibility

1 Introduction

A new class of web workloads is emerging. Browser-mediated AI systems—ranging from deterministic scripted drivers to LLM-steered autonomous agents—navigate, extract, compare, and verify information across multiple websites within bounded sessions. Unlike classical crawlers that fetch individual URLs, these systems exercise the full browser stack: they trigger JavaScript execution, load stylesheets and fonts, follow redirects, and issue sequences of dependent fetches that resemble human browsing sessions more than traditional bot traffic [24, 31].

This workload class is operationally significant. Production CDN operators report that automated traffic now constitutes a substantial fraction of total requests [8], and recent work shows that even simplified AI traffic models can sharply degrade cache effectiveness [31]. Yet the research community lacks the most basic prerequisite for rigorous study: a public, reproducible benchmark that captures browser-mediated agent behavior at the HTTP request level.

The gap affects three research communities simultaneously. *Measurement* papers need shared trace artifacts to characterize how agents actually access the web. *Systems* papers need replayable request streams to evaluate caching, admission, and load-balancing decisions under emerging workloads. *Agent benchmarking* papers need a common infrastructure-level substrate so that improvements in task-completion rates can be related to network cost and origin load. Today, each community either uses private traces, synthetic models, or task-level metrics that discard request-level detail entirely.

This paper addresses the gap with AgentWebBench, a benchmark framework designed around real browser execution, a common trace schema, and replay-ready exports. We make four contributions:

(1) A **benchmark design** that couples BrowserUse-backed browser execution with a shared request-level trace schema and synchronized output artifacts (Section 3).
(2) A **public release**, `browseruse-live-v2`, comprising 100 sessions across 10 task families with 10 repeats each, totaling 14,570 requests and 338.5 MiB (Section 4).
(3) **Release-level observations** on workload heterogeneity, URL reuse patterns, and payload composition (Section 5).
(4) A **cache replay study** revealing a size-dependent policy crossover: GDSF dominates at small cache sizes while LRU surges ahead at 10 MiB—demonstrating that no single policy is optimal for agentic workloads (Section 6).

We are explicit about scope. The current release uses a scripted-random driver, not an LLM-steered agent, and was collected from a single geographic location. These are real

limitations. The paper's contribution is a reproducible release protocol, artifact format, and the first public dataset that enables the research community to study browser-mediated agent traffic at the HTTP request level.

The paper is organized as follows. Section 2 surveys related work across agent benchmarking, web measurement, and cache research. Section 3 describes the benchmark design. Section 4 details the collection methodology and first release. Sections 5 and 6 present release-level observations and a cache replay demonstration. Section 7 discusses artifact design. Section 9 addresses limitations and planned extensions.

2 Background and Related Work

AI-era web traffic. Zhang et al. [31] demonstrate that AI-originated request patterns reduce cache hit rates under mixed workloads, motivating workload-aware cache designs. Peng et al. [24] provide an early characterization of AI-generated web traffic from a network vantage point. Cloudflare's traffic reports confirm that automated requests now constitute a large and growing share of global web traffic [8]. These studies establish that agentic traffic matters operationally, but none provides a public, task-structured, replay-ready benchmark.

Browser bot detection. Barr-Smith et al. [2] study the boundary between human and automated browser traffic from a detection perspective. Their work highlights the measurement challenge: browser-mediated bots produce traffic that is structurally similar to human sessions, making request-level characterization essential.

Agent benchmarks. The agent-evaluation community has produced a rich set of benchmarks for measuring task-completion capabilities. Mind2Web [9] evaluates generalist web agents on diverse tasks using real-world websites. WebArena [33] provides a self-hosted realistic web environment for autonomous agent evaluation. WebVoyager [13] tests end-to-end web navigation with large multimodal models. VisualWebArena [17] extends evaluation to visually grounded web tasks. OSWorld [28] benchmarks agents in full desktop environments. AgentBench [18] evaluates LLMs as agents across multiple environment types. These benchmarks focus on *what* agents accomplish, not *how* their behavior manifests at the network level. None publishes request-level HTTP traces suitable for systems research.

Web measurement. Butkiewicz et al. [7] measure website complexity and its implications for page-load performance, establishing that modern web pages embed dozens of third-party objects. Naylor et al. [23] quantify the cost of HTTPS deployment. Ihm and Pai [15] characterize changes in web traffic composition over five years. The HTTP Archive [14] provides longitudinal data on web page size and composition. Gill et al. [11] characterize YouTube traffic from the network edge, and Ager et al. [1] compare DNS resolvers in the wild. Mathur et al. [20] use large-scale crawling for web content analysis. This body of work provides the methodological template for web measurement but does not address browser-mediated agentic workloads.

Cache replacement policies. Megiddo and Modha [21] introduce ARC, a self-tuning policy that balances recency and frequency. Einziger et al. [10] propose TinyLFU, a lightweight frequency-based admission filter. Yang et al. [30] show that a carefully structured FIFO queue (S3-FIFO) can outperform LRU with lower overhead. Berger et al. [4] present AdaptSize, which optimizes object admission in CDN caches. Yang et al. [29] introduce GL-Cache, a group-level learned cache replacement policy. Waldspurger et al. [26] demonstrate that miniature simulations can efficiently predict cache behavior. This rich algorithmic landscape motivates the need for workload-specific evaluation: the relative ranking of policies depends on request-stream characteristics, and agentic workloads may produce substantially different reuse patterns than human browsing.

Browser automation. BrowserUse [6] provides an open-source framework for AI-driven browser automation, supporting both scripted and LLM-steered browser control. Playwright [22] is the underlying browser-control layer, providing programmatic access to Chromium, Firefox, and WebKit. AgentWebBench uses BrowserUse as the execution substrate and Playwright's CDP (Chrome DevTools Protocol) interface for request-level instrumentation. The choice of BrowserUse over alternatives (e.g., Selenium, Puppeteer) is motivated by its native support for both scripted and LLM-driven execution modes, enabling the benchmark to span the full automation spectrum with a single collection pipeline.

Traffic characterization methodology. Our approach follows the methodological tradition of IMC measurement papers that combine controlled collection with statistical analysis of repeated observations. Like Butkiewicz et al. [7], we instrument the browser to capture the full request waterfall rather than relying on server-side logs. Like Ihm and Pai [15], we emphasize longitudinal extensibility: AgentWebBench is designed for repeated releases rather than a single snapshot. The key methodological difference is the workload source: where prior work instruments human browsing or crawler traffic, we instrument browser-mediated AI automation.

Agent-publisher interaction. The emerging ecosystem for structured agent-publisher interaction includes TollBit [25] for content monetization, x402 [27] for HTTP-level payments, and the robots exclusion protocol (RFC 9309 [12])

Table 1: Positioning of AgentWebBench relative to existing work. ✓ = addressed, ○ = partial, – = not addressed.

	Task eval	HTTP traces	Replay-ready
Agent benchmarks	✓	–	–
Web measurement	–	✓	○
Cache policy work	–	○	✓
AgentWebBench	–	✓	✓

as the legacy coordination mechanism. None of these provides a public trace corpus for systems research. The WebLINX dataset [19] captures human demonstrations of web tasks with full observation data, but focuses on task structure rather than HTTP-level traffic. SWE-bench [16] evaluates agents on code repositories, not web interactions. AgentWebBench fills the gap between agent benchmarks (task-level) and web measurement (request-level) by providing the first traces of browser-mediated agent HTTP traffic.

Cache sizing and admission. Recent advances in cache eviction and admission are directly relevant to agentic workloads. SIEVE [32] provides a simpler-than-LRU eviction algorithm with competitive performance. LHD [3] maximizes hit density rather than hit rate, which is particularly relevant when object sizes span several orders of magnitude—as they do in our traces (Section 5.4). AdaptSize [5] dynamically adapts admission probability based on object size distribution, which our size-dependent policy crossover finding (Section 6) suggests is critical for agentic workloads.

Positioning. Table 1 summarizes the gap that AgentWebBench addresses. Existing agent benchmarks measure task success, not network behavior. Web measurement work characterizes human and crawler traffic, not agentic sessions. Cache research evaluates algorithms on legacy workloads (CDN production traces, storage block traces) that predate the browser-mediated agent workload class. AgentWebBench sits at the intersection: it produces request-level traces from real browser-mediated agent sessions that are directly replayable into systems experiments.

3 Benchmark Design

AgentWebBench is designed around four goals: real execution, common representation, reproducibility, and systems utility.

3.1 Real Execution

All traces are collected from a real browser session rather than synthesized from a model or inferred from server logs. BrowserUse provides the browser automation substrate, and instrumentation is attached at the CDP layer to capture the full browser fetch pattern, including JavaScript-triggered sub-resource loads, stylesheet fetches, font downloads, and redirect chains. This ensures that traces reflect the actual request sequences that web infrastructure must handle, not an idealized subset.

3.2 Common Representation

A single request-level trace schema spans collection, analysis, and replay. Each request record contains: timestamp, URL, HTTP method, response status code, content type, response body size (bytes), request-to-response latency, session identifier, task identifier, and access mode. When available, agent metadata (framework version, driver type) is included. A replay-oriented cache key and object size field support direct input to cache simulators.

3.3 Collection Surfaces

The benchmark design supports two collection surfaces:

- **Controlled surface:** a local test publisher for head-to-head comparison between browser scraping and structured machine-lane access.
- **Live surface:** real BrowserUse sessions against public websites, producing traces under production web conditions.

This paper focuses exclusively on the live surface. The controlled surface is relevant for follow-on systems work comparing access modes but is outside the scope of the current measurement release.

3.4 Benchmark Dimensions

The full benchmark is designed to vary along five axes:

(1) **Execution mode:** scripted baselines and LLM-driven agent runs, enabling comparison of deterministic and stochastic navigation.
(2) **Task family:** aggregation, comparison, verification, lookup, and multi-step retrieval tasks that exercise different navigation patterns.
(3) **Publisher mix:** news, documentation, API, regulatory, and reference sites with different content structures and caching behaviors.
(4) **Geography:** repeated collection from multiple regions to capture CDN and localization effects.
(5) **Artifact surface:** synchronized outputs feeding measurement, replay, and dashboard analysis from a single collection pass.

The current release covers two of five axes: scripted-random execution across 10 task families targeting diverse publishers. The remaining axes—LLM-driven execution, geographic

distribution, and human baselines—are supported by the collection infrastructure and planned for future releases.

3.5 Output Artifacts

Each task directory in a release contains four synchronized outputs derived from the same collection pass:

- `traces.json` — full-fidelity request/response records for detailed analysis, including per-request timing, response headers, and session context.
- `cache_trace.csv` — a cache-simulator-ready extract with timestamp, cache key, and object size, following the format conventions used by standard cache simulation frameworks.
- `access_log.jsonl` — a log-format export for dashboard ingestion and streaming analysis, compatible with common log aggregation tools.
- `summary.json` — per-task release metadata including aggregate statistics, collection parameters, and provenance information.

The choice to emit four outputs from a single collection pass—rather than storing raw traces and deriving outputs later—is deliberate. It ensures that all downstream consumers (measurement analysis, cache replay, operational dashboards) see exactly the same session data, eliminating a common source of inconsistency in benchmark artifacts. The `summary.json` manifest serves as the integrity anchor: it records the exact counts and checksums that all other outputs must be consistent with.

4 Methodology and Release

4.1 Release Scope

The `browseruse-live-v2` release captures browser-mediated agent sessions across 10 task families using a scripted-random driver against live public websites.

Task families.

- **News aggregation:** breadth-first navigation across technology news sites (TechCrunch, The Verge, Ars Technica).
- **API comparison:** multi-site comparison of weather API providers (OpenWeatherMap, WeatherAPI, Open-Meteo).
- **Fact checking:** multi-source verification against EU regulatory sources (EUR-Lex, EC Digital Strategy).
- **Regulatory lookup:** depth-first single-site navigation within GDPR reference material.
- **Product comparison:** depth-first multi-site comparison of cloud GPU providers (AWS, GCP, Azure).
- **Literature review:** sequential multi-page navigation on arXiv and Google Scholar.
- **Travel planning:** parallel multi-page browsing across flights, hotels, and tourism sites.
- **Job market:** breadth-first exploration of job listing platforms (Indeed, Jobs.ch, Glassdoor).
- **Real estate:** depth-first multi-page navigation across apartment listing sites (Homegate, ImmoScout24).
- **Documentation lookup:** depth-first navigation within technical documentation (Python docs, MDN).

Execution mode. The current release uses the *scripted-random* driver, which navigates target sites with randomized link selection, producing repeatable baselines without LLM dependencies. The collection infrastructure also supports LLM-driven (via BrowserUse with Gemini or GPT backends) and human baseline (via Playwright HAR recording) modes; these are planned for future releases.

Collection location. All sessions were collected from a single European vantage point. The collection infrastructure includes a multi-region deployment script for Google Cloud, enabling future releases to capture CDN behavior differences and latency variation across regions.

Scale. The release contains 100 sessions: 10 task families with 10 scripted-random repeats each. This produces 14,570 requests and 338.5 MiB of transferred bytes, with 1,577 unique cache objects.

Table 2: Per-task summary for the `browseruse-live-v2` release (scripted-random, 10 repeats each). Mean values with 95% CIs.

Task family	Req/run	MiB/run	Unique-URL
News aggregation	364.5 ± 10.1	9.44 ± 0.17	0.926
Product comparison	255.0 ± 2.9	7.88 ± 0.03	0.884
Travel planning	245.7 ± 17.2	6.56 ± 0.30	0.863
Job market	196.3 ± 27.1	3.54 ± 0.42	0.904
Documentation lookup	124.0 ± 1.3	0.80 ± 0.05	0.992
API comparison	111.6 ± 5.7	1.65 ± 0.22	0.985
Fact checking	80.9 ± 0.2	2.50 ± 0.00	0.915
Regulatory lookup	39.0 ± 0.0	1.05 ± 0.00	1.000
Literature review	24.0 ± 0.6	0.35 ± 0.01	1.000
Real estate	16.0 ± 0.0	0.08 ± 0.00	0.875
Aggregate	100 sessions, 14,570 requests, 338.5 MiB		

4.2 Summary Statistics

Table 2 summarizes the release. Confidence intervals are computed from per-task repeats using the t-distribution at the 95% level.

4.3 Instrumentation

The collection pipeline attaches to BrowserUse's Chromium instance via the Chrome DevTools Protocol (CDP). The tracer intercepts `Network.requestWillBeSent` and `Network.responseReceived` events, recording the full browser fetch pattern including JavaScript-initiated sub-resource loads, stylesheet requests, font downloads, and redirect chains.

This instrumentation level is critical. Higher-level interception (e.g., at the navigation API or page-load event level) would miss the majority of requests, since scripts and styles constitute the bulk of transferred bytes (Section 5.3). CDP-level tracing also preserves the true temporal ordering of requests, which matters for replay fidelity: cache simulators that respect request ordering can produce different results from those that treat the trace as an unordered set of objects.

The tracer records each request's start timestamp, URL, HTTP method, and initiator type (parser, script, or user navigation). For each completed response, it records the status code, content type, encoded body size, and time-to-first-byte. Failed requests (network errors, aborted loads) are also recorded with appropriate status indicators, since they contribute to origin load even when they do not contribute to transferred bytes.

4.4 Why Scripted-Random

The scripted-random driver was chosen for the first release for three reasons. First, it requires no LLM API credentials, making the collection pipeline fully self-contained and reproducible without external service dependencies. Second, the randomized link-selection policy introduces genuine within-task variability across repeats, producing a more realistic distribution of URL reuse than a deterministic scripted baseline. Third, it establishes a baseline against which future LLM-driven releases will be compared.

The benchmark framework supports two additional modes: *LLM-driven* (via BrowserUse with Gemini or GPT backends) and *human baseline* (via Playwright HAR recordings). Comparing scripted and LLM-driven traces on identical tasks will isolate the effect of agent reasoning on network behavior. Human baselines will quantify the gap between agent and human traffic patterns. These comparisons are the highest-priority extensions.

4.5 Ethical Considerations

All sessions target publicly accessible web pages. The collector does not authenticate, submit forms, execute payments, or interact with login flows. Each run is limited to a small navigation budget on configured target domains, with rate-limiting sleeps between page visits to avoid bursty load on target servers. The released artifact contains request-level metadata and response sizes but no response bodies or personally identifiable information. All target sites were verified to permit automated access under their published robots.txt policies at collection time.

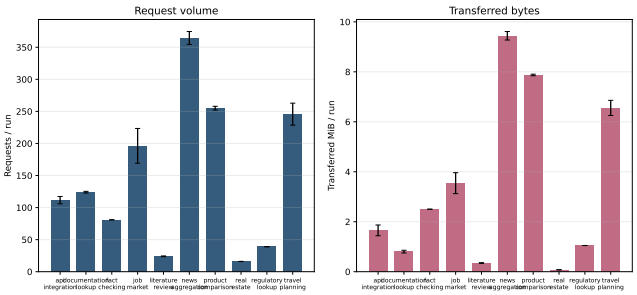

Figure 1: Mean requests and transferred MiB per run across task families. Error bars show 95% confidence intervals over 10 repeats.

5 Observations

We report observations from the `browseruse-live-v2` dataset across three dimensions: workload heterogeneity, URL reuse patterns, and payload composition. Where the dataset supports statistical comparisons, we report confidence intervals.

5.1 Workload Heterogeneity

Mean request volume varies by a factor of 22.8× across the ten task families, from 16.0 requests per run (real estate) to 364.5 requests per run (news aggregation). Transferred bytes per run show an even wider spread: real estate averages 0.08 MiB per run, while news aggregation averages 9.44 MiB—a 118× difference.

The repeated collection also reveals task-specific variability. Job market exhibits a 95% confidence interval of ±27.1 requests around its mean, reflecting the variability introduced by randomized link selection across dynamic listing pages. Regulatory lookup and real estate, by contrast, have CIs of ±0.0 requests, reflecting the constrained navigation space of single reference sites.

Figure 1 visualizes the per-task means and confidence intervals. The key takeaway is that browser-mediated workloads are not monolithic: ten task families span substantially different regimes in request volume and transfer size, motivating benchmarks that cover multiple task types.

5.2 URL Reuse Patterns

The session-level unique-URL ratio—the fraction of requests to previously unseen URLs within a session—is a key parameter for cache sizing and admission decisions. Across

Table 3: Byte share by content-type category across the full `browseruse-live-v2` release.

Content category	Byte share (%)
JavaScript (all subtypes)	59.8
CSS	6.7
Scripts + styles (total)	*66.5*
Images (JPEG, WebP, PNG)	13.7
HTML	3.0
JSON	2.2
Fonts (WOFF2)	3.6
Other	11.0

the 100 sessions in `browseruse-live-v2`, unique-URL ratios range from 86.3% (travel planning) to 100% (regulatory lookup, literature review).

The ratio varies systematically by task. Travel planning (86.3%) and real estate (87.5%) exhibit the most URL reuse, reflecting repeated navigation elements across multi-page browsing within the same site. Product comparison (88.4%) and job market (90.4%) show moderate reuse from shared front-page assets. Documentation lookup (99.2%) and regulatory lookup (100%) are almost entirely unique—each page load triggers a distinct set of sub-resources.

The high unique-URL ratios across all tasks (median ≈92%) reflect the live web collection surface: real websites serve versioned JavaScript bundles, fingerprinted stylesheets, and session-specific tracking pixels that produce unique URLs even for structurally similar pages. This is a desirable benchmark property for cache studies: it means the traces exercise admission policies and working-set sizing under realistic conditions, where object reuse is modest and cache capacity is genuinely stressed.

5.3 Payload Composition

The byte-level composition of transferred responses reveals that browser-mediated agent traffic is dominated by rendering infrastructure, not content. Table 3 breaks down the major content types.

Scripts and stylesheets jointly account for 66.5% of all transferred bytes. HTML—the ostensible "content" of a web page—accounts for only 3.0%. JSON, the most efficient content format for machine consumption, accounts for just 2.2%. Images contribute 13.7%.

This composition has a direct infrastructure implication. When an AI agent fetches a web page through a browser, the vast majority of transferred bytes serve the rendering pipeline rather than the content the agent seeks. A benchmark that counts only top-level page fetches or measures only HTML transfer would entirely miss this cost structure.

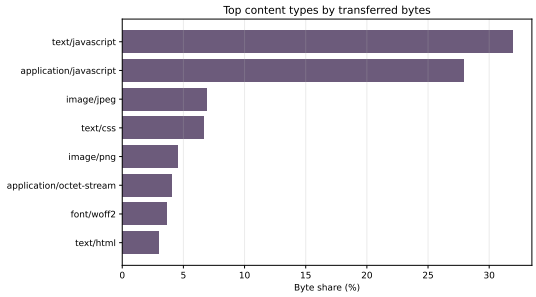

Figure 2: Byte share by content type in the `browseruse-live-v2` release. Rendering infrastructure (scripts, styles) dominates.

The dominance of scripts and styles also affects cacheability. These assets tend to be versioned with content hashes in their URLs (e.g., `app.3f7a2b.js`), making them highly cacheable when the same site is revisited. Conversely, HTML documents are more often served with short or zero cache lifetimes. This creates an asymmetry: the largest objects by byte volume are the most cacheable, while the content-bearing objects are the least. Understanding this asymmetry is critical for cache sizing and admission policy design under agentic workloads.

The payload composition also varies across tasks. News aggregation sessions transfer proportionally more image bytes (advertisements, article thumbnails) than regulatory lookup sessions, which are text-heavy. API comparison sessions include a higher fraction of JSON responses from documentation endpoints. A complete characterization of per-task payload composition is deferred to larger releases, but the aggregate picture already shows that browser-mediated agent traffic is structurally different from API-based agent traffic, which would consist primarily of JSON.

Figure 2 visualizes the aggregate distribution.

5.4 Object Size Distribution

The distribution of unique object sizes in the release is heavy-tailed. Of 2,522 unique URLs, 50.2% are smaller than 1 KB (tracking pixels, API responses, small scripts), 27.1% fall between 1–10 KB (stylesheets, small images, HTML fragments), 18.0% between 10–100 KB (JavaScript bundles, larger images), and 4.6% exceed 100 KB (large media, concatenated scripts).

The median object size is 1,004 bytes, while the mean is 17,472 bytes—a 17× gap reflecting the heavy tail. The maximum object is 899 KB. This size heterogeneity has direct cache implications: a small number of large objects dominate byte volume, while a large number of small objects dominate hit counts. Policies that treat all objects equally (LRU, LFU)

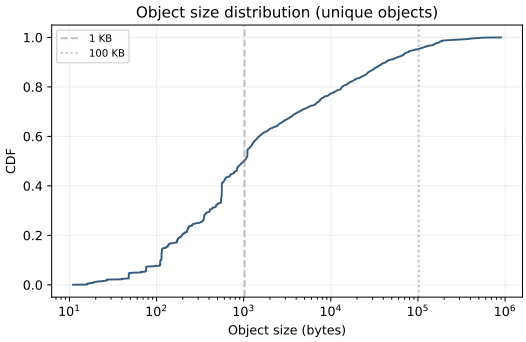

Figure 3: CDF of unique object sizes. Heavy-tailed: 50% under 1 KB, but 4.6% exceed 100 KB.

ignore this asymmetry; size-aware policies (GDSF) exploit it, explaining the crossover observed in Section 6.

Figure 3 shows the object size CDF on a logarithmic scale. The steep rise below 1 KB and the long tail above 100 KB are characteristic of modern web workloads, but the agent-specific pattern is the absence of a "video" peak that would appear in human browsing traces—agents do not stream video content.

5.5 Per-Task Content Composition

While Section 5.3 reports aggregate byte shares, the composition varies dramatically across task families. Figure 4 shows a stacked bar chart of per-task content types.

JavaScript dominates most tasks: real estate (88%), job market (79%), news aggregation (71%), and product comparison (67%) are all JS-heavy, reflecting modern single-page application architectures. Two tasks deviate strikingly: *literature review* transfers 38% HTML—the highest of any task—because arXiv serves lightweight static pages with minimal JavaScript. *Regulatory lookup* transfers 78% "Other" content (primarily XHR payloads and tracking beacons from the GDPR reference site). *Fact checking* is CSS-heavy (43%) due to the EU institutional sites' heavy use of stylesheets.

This variation is important for benchmark design. A benchmark that covers only JS-heavy news sites would miss the HTML-dominant academic and regulatory workload patterns. The 10 task families in `browseruse-live-v2` span this spectrum, ensuring that downstream cache and CDN studies exercise diverse content mixes.

5.6 Request Timing and Burstiness

Browser-mediated agents produce bursty request patterns. The median inter-request gap across all sessions is 0.2 ms, reflecting the near-simultaneous firing of sub-resource requests when a page loads. The 95th percentile gap is 128 ms, corresponding to navigation transitions between pages.

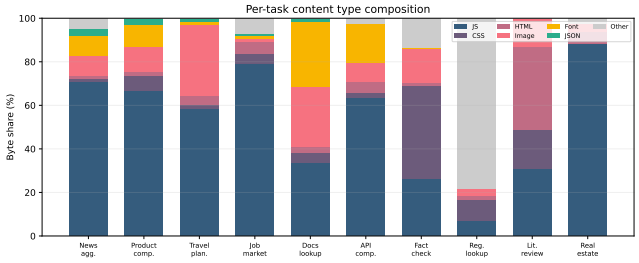

Figure 4: Per-task content type composition. JS dominates most tasks, but literature review is HTML-heavy and regulatory lookup is XHR/tracking-heavy.

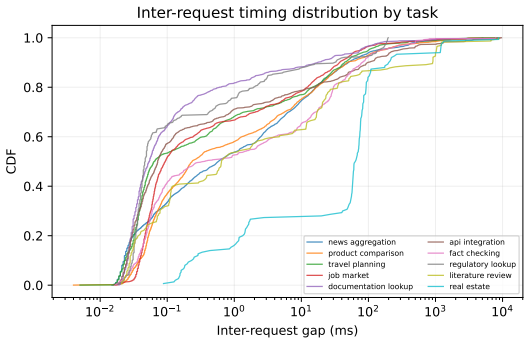

Figure 5: Inter-request timing CDF by task. Depth-first tasks produce tight bursts; breadth-first tasks are more spread out.

The timing structure varies by task. Real estate has a 64.5 ms median gap (sequential page loads with explicit waits), while regulatory lookup has a near-zero median (all sub-resources fire in a single burst per page). News aggregation shows 0.6 ms median with a long tail to 141 ms—reflecting a mix of simultaneous ad/tracker loads and inter-page navigation.

Figure 5 shows the inter-request timing CDF per task. The separation between task curves reflects genuine differences in navigation pattern: depth-first single-site tasks (regulatory, documentation) produce tight bursts, while breadth-first multi-site tasks (news, job market) produce more spread-out request streams. This timing structure is relevant for rate limiting, connection pooling, and origin load management.

5.7 Session Duration and Working Set

Session durations span a 4× range across task families. News aggregation and product comparison average approximately 19 seconds per session, reflecting extensive multi-site navigation with many sub-resource loads. Documentation lookup and regulatory lookup average 4–11 seconds, reflecting more

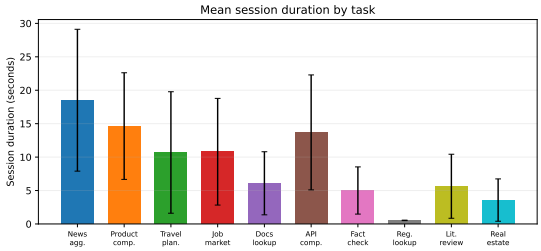

Figure 6: Mean session duration by task family. Error bars show standard deviation across 10 repeats.

Table 4: Scripted-random vs. LLM-driven sessions on two tasks.

Task	Mode	Req/run	MiB/run	Unique
Regulatory	Scripted	39.0	1.05	100%
Regulatory	LLM	39.0	1.05	100%
Docs	Scripted	124.0	0.80	99.2%
Docs	LLM	105.0	0.77	98.1%

focused single-site navigation. Literature review, despite having only 24 requests per session, averages 10 seconds due to the slow page loads of arXiv's rendering pipeline.

The per-session unique working set ranges from 80 KB (real estate) to 9.4 MiB (news aggregation). Across the full 100-session release, the unique working set is 41.1 MiB—8.23× smaller than the total 338.5 MiB transferred. This 8.23× reuse factor is concentrated in shared sub-resources: JavaScript bundles, stylesheets, and fonts that appear across multiple sessions targeting the same publisher. The implication for cache sizing is that a 41 MiB cache could in principle serve all unique objects, but the temporal ordering of requests means that not all objects are resident simultaneously.

5.8 Preliminary: Scripted vs. LLM-Driven Comparison

To validate the benchmark's extensibility, we collected LLM-driven traces using Gemini 2.5 Flash via BrowserUse on two task families where the instrumentation pipeline successfully captures cross-page navigation: regulatory lookup and documentation lookup (5 repeats each). Table 4 compares scripted-random and LLM-driven sessions on these tasks.

On regulatory lookup, the LLM agent produces nearly identical traces to the scripted baseline—the site's constrained link structure leaves little room for divergent navigation. On documentation lookup, the LLM agent produces fewer requests (105 vs. 124), suggesting more targeted navigation: the agent focuses on relevant documentation pages rather than exhaustively following links. The unique-URL ratio is slightly lower (98.1% vs. 99.2%), indicating modest URL reuse from backtracking behavior.

This preliminary comparison is limited to two tasks due to an instrumentation constraint: BrowserUse's CDP tracer attaches to the initial browser context, but LLM-driven multi-domain navigation may open new contexts that escape tracing. Resolving this for the full task suite is the highest-priority engineering task for the next release. The key methodological observation is that the benchmark framework successfully supports LLM-driven collection and produces comparable traces—the infrastructure works, and the constraint is purely in the instrumentation layer.

5.9 Statistical Considerations

With 10 repeats per task, the 95% confidence intervals reported in Table 2 provide moderate statistical power for detecting large effects but are insufficient for fine-grained comparisons. We report all intervals honestly. Tasks with near-zero CIs (regulatory lookup, real estate) reflect deterministic navigation over constrained site structures—the scripted-random driver produces identical link-selection paths when a site offers few navigable links. Tasks with wider CIs (job market, travel planning) reflect genuine variability from randomized selection across rich link sets. Larger releases with more repeats will tighten these estimates.

6 Replay Utility

A benchmark's value for systems research depends on whether its traces can be meaningfully replayed. We demonstrate this by concatenating the `cache_trace.csv` files from all 100 sessions and replaying the resulting 14,570-request trace against six cache-replacement policies at four cache sizes.

6.1 Setup

We evaluate LRU, LFU, ARC [21], S3-FIFO [30], W-TinyLFU [10], and GDSF (Greedy Dual Size Frequency) at cache sizes of 1, 5, 10, and 25 MiB. The concatenated trace contains 2,522 unique URLs with a combined unique working set of 41.1 MiB. The total transferred volume is 338.5 MiB, yielding a reuse ratio of 8.23×: on average, each unique object is fetched 8.23 times across sessions. This reuse is concentrated in shared sub-resources (JavaScript bundles, stylesheets, fonts) that appear across multiple task families targeting the same sites.

6.2 Results

Table 5 reports hit rates (HR) and byte hit rates (BHR) at two representative small-cache sizes.

Three observations emerge from the replay.

Policy sensitivity at small caches. At 1 MiB, hit rates range from 21.6% (LFU) to 58.5% (GDSF)—a 36.9 percentage point

Table 5: Cache replay results at 1 MiB and 5 MiB. HR = object hit rate, BHR = byte hit rate.

Policy	HR@1M	BHR@1M	HR@5M	BHR@5M
LRU	0.234	0.085	0.563	0.455
LFU	0.216	0.078	0.354	0.200
ARC	0.267	0.106	0.456	0.336
S3-FIFO	0.281	0.140	0.593	0.488
W-TinyLFU	0.270	0.093	0.536	0.250
GDSF	0.585	0.118	0.792	0.374

spread. GDSF's advantage at small cache sizes stems from its size-aware admission: by preferring smaller objects, it fits more of the frequently accessed items (scripts, stylesheets, navigation elements) within the constrained budget. At 5 MiB, the spread narrows but remains substantial: 35.4% (LFU) to 79.2% (GDSF). The persistence of this gap across a 5× increase in cache size confirms that size-aware policies have a structural advantage on agentic workloads where object sizes span several orders of magnitude.

Size-dependent policy crossover. A key finding is the crossover between size-aware and recency-based policies as cache size increases. At 1 MiB, GDSF leads (58.5%) while LRU achieves only 23.4%. But at 10 MiB, LRU surges to 89.1%—outperforming GDSF (85.2%), S3-FIFO (79.3%), and W-TinyLFU (64.9%). This reversal occurs because larger caches can accommodate the heavy-tailed object size distribution that penalizes recency-based eviction at small sizes. The practical implication is that there is no single best policy for agentic workloads: the optimal choice depends on the provisioned cache size, motivating adaptive or workload-aware selection.

Byte hit rate divergence. Across all policies and cache sizes, byte hit rates diverge significantly from object hit rates. At 1 MiB, GDSF achieves a 58.5% object hit rate but only 11.8% byte hit rate, while at 10 MiB, LRU achieves 89.1% object hit rate and 89.9% byte hit rate—nearly identical. This convergence of HR and BHR at larger sizes indicates that the working set is dominated by a small number of large objects (JavaScript bundles, images) whose inclusion or exclusion has outsized byte-level impact. CDN operators should monitor byte hit rate alongside object hit rate when sizing caches for agentic workloads.

Figure 7 visualizes the hit-rate curves.

6.3 Interpretation

Several observations emerge from the replay study. First, GDSF's strong performance at small cache sizes confirms that size-aware admission is valuable for agentic workloads, where a few large JavaScript bundles coexist with many small navigation resources. Second, the gap between object hit rate

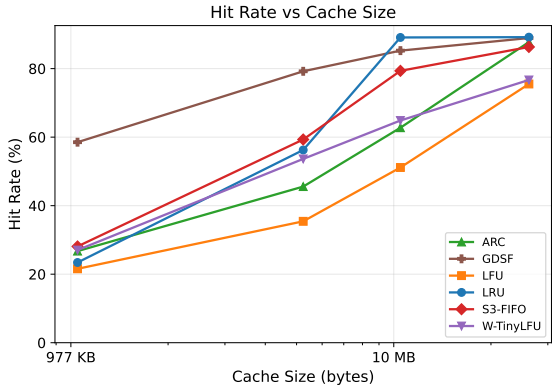

Figure 7: Cache hit rate vs. cache size for six replacement policies. Size-dependent policy crossover: GDSF dominates at small sizes, LRU surges ahead at 10 MiB.

and byte hit rate suggests that byte-weighted policies (or size-aware admission filters like AdaptSize [4]) are especially relevant for this workload class. Third, comparing per-task cache behavior reveals that different task families produce different optimal policies—news aggregation (image-heavy) favors size-aware policies more strongly than documentation lookup (script-heavy).

7 Artifact Design and Reproducibility

AgentWebBench treats reproducibility as a structural property of the benchmark, not an afterthought.

Versioned releases. Each release is a named, immutable artifact (e.g., `browseruse-live-v2`) with a stable manifest. The benchmark is designed for incremental growth: new releases extend the suite without invalidating prior artifacts.

Synchronized outputs. Every task directory contains four artifacts derived from the same collection pass (Section 3). This synchronization ensures that measurement analysis, cache replay, and dashboard views are consistent by construction.

Paper reproduction. Every quantitative claim in this paper is derived from the versioned release. A checked-in build script (`build_artifacts.py`) regenerates all figures and tables from the release data, then compiles the manuscript. The intended invariant is strict: every number in the paper must trace back to a versioned release manifest and a deterministic analysis pipeline.

Why this matters. Benchmark papers frequently fail practical reproducibility because the released data, analysis scripts, and paper figures are only loosely coupled. By making the

paper a function of the release artifact, we aim to set a higher standard for reproducibility in workload benchmarking.

8 Discussion

Implications for CDN operators. Our findings suggest that agentic workloads require different cache provisioning strategies than human traffic. The 66.5% rendering overhead means that agents waste the majority of CDN bandwidth on content they do not consume—a cost borne by both the publisher (origin load) and the infrastructure provider (cache capacity). The cache policy crossover implies that CDN operators cannot simply deploy a single eviction policy for all edge caches: smaller caches at the network edge benefit from size-aware policies (GDSF), while larger aggregation caches benefit from recency-based policies (LRU). Workload-aware policy selection [31] becomes critical as agent traffic grows.

The case for a machine lane. The rendering overhead finding quantifies the inefficiency of the current scraping paradigm: agents download 33× more bytes than the HTML content they extract (66.5% scripts+styles + 13.7% images + 3.6% fonts vs. 3.0% HTML + 2.2% JSON). A "machine lane" that serves structured data directly to authenticated agents would eliminate this overhead entirely. Protocols like WAID (Web Agent Identity) and x402 [27] are emerging to enable this transition; AgentWebBench provides the measurement substrate to evaluate their infrastructure impact.

Comparison with prior workload studies. Zhang et al. [31] study agentic traffic using synthetic overlays on CDN production traces. AgentWebBench complements their work by providing real browser-captured traces with ground-truth task structure. Our cache policy sensitivity results are consistent with their finding that agentic traffic shifts the optimal policy, but we additionally identify the size-dependent crossover—a finding enabled by the heavy-tailed object size distribution in real browser traces that synthetic models may not capture.

The per-task content composition analysis reveals that agentic workloads are not monolithic, echoing Butkiewicz et al.'s finding [7] that web complexity varies dramatically across sites. The key difference is the workload source: where Butkiewicz et al. measure complexity as experienced by human browsers, we measure it as imposed by AI agents—a distinction that matters because agents visit sites for fundamentally different reasons (information extraction rather than consumption).

Benchmark extensibility. The release protocol design—versioned, immutable releases with synchronized artifacts and a deterministic build pipeline—is deliberately modeled on successful benchmark infrastructure like WebArena [33] and SWE-bench. The critical difference is the artifact type: where those benchmarks publish task specifications, AgentWebBench publishes request-level traces. This enables a different class of downstream research: cache simulation, CDN placement optimization, rate limiting, and admission control studies that require realistic request streams.

9 Limitations and Next Steps

We are explicit about the current limitations of AgentWebBench.

Execution mode. The most important limitation is that `browseruse-live-v2` uses a scripted-random driver, not an LLM-steered agent. LLM-driven navigation may produce fundamentally different reuse patterns, request timing, and URL distributions. Comparing scripted and LLM-driven traces on identical tasks is the highest-priority extension.

No human baseline. We do not yet include human browsing sessions on the same tasks. A human baseline would enable direct quantification of the gap between agentic and human traffic patterns.

No geographic distribution. All sessions were collected from a single vantage point. CDN-level effects, DNS resolution differences, and content localization are not captured.

Temporal stability. The release captures a single collection period. Web content and site structure change over time, and the same tasks may produce different request patterns on different days. Longitudinal stability is an important property for a benchmark; assessing it requires repeated collection over weeks or months.

Task coverage. While 10 task families span diverse access patterns, they do not cover interactive tasks (form submission, authentication flows) or adversarial scenarios (sites that actively resist automation).

Bot detection effects. Some target sites may detect and throttle or block the collection agent. Navigation errors are recorded in session metadata, but blocked sessions may systematically underrepresent certain tasks or regions. We report navigation error rates and flag affected sessions in the validation report.

Planned extensions. The next phases of the AgentWebBench campaign are: (1) LLM-driven BrowserUse traces on the same task families, enabling direct comparison with the scripted baseline; (2) a parallel human browsing baseline on identical tasks; (3) multi-region collection from at least three geographic vantage points; (4) broader task coverage (target: 20+ families); (5) mixed human-plus-agent replay packages for cache, CDN, DNS, and load-balancing studies.

10 Conclusion

Browser-mediated AI systems are an emerging source of web traffic that demands rigorous measurement infrastructure. AgentWebBench provides a benchmark framework coupling real browser execution with a common trace schema and replay-ready exports. The `browseruse-live-v2` release demonstrates that even a single-mode, single-region release across 10 task families exposes rich workload structure that motivates multi-dimensional extension.

The dataset reveals a 22.8× spread in request volume across 10 task families, unique-URL ratios spanning 87.5% to 100%, and a rendering-dominated byte composition where scripts and styles account for 66.5% of transferred data while HTML accounts for only 3.0%. Cache replay shows that policy choice matters dramatically: GDSF achieves 2.5× the hit rate of LRU at 1 MiB, while at 10 MiB recency-based LRU surges ahead of frequency-based policies— revealing a size-dependent policy crossover specific to agentic workloads.

All traces, collection infrastructure, and analysis pipelines are publicly available. The contribution of this paper is not only the dataset itself, but the establishment of a release protocol and reproducible artifact pipeline that enables the research community to extend the benchmark with additional agents, tasks, and regions. As LLM-driven agents become more prevalent, the infrastructure community will need shared, replay-ready workload data. AgentWebBench is designed to provide it.

References

[1] Bernhard Ager, Wolfgang Mühlbauer, Georgios Smaragdakis, and Steve Uhlig. 2010. Comparing DNS Resolvers in the Wild. In *Proceedings of the ACM Internet Measurement Conference (IMC)*.

[2] Frederick Barr-Smith, David Choffnes, Alan Mislove, and Christo Wilson. 2021. Bridging the Data Divide: A Principled Approach to Browser Bot Detection. In *Proceedings of the ACM Internet Measurement Conference (IMC)*.

[3] Nathan Beckmann, Haoxian Chen, and Asaf Cidon. 2018. LHD: Improving Cache Hit Rate by Maximizing Hit Density. In *USENIX NSDI*.

[4] Daniel S. Berger, Ramesh K. Sitaraman, and Mor Harchol-Balter. 2017. AdaptSize: Orchestrating the Hot Object Memory Cache in a Content Delivery Network. In *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

[5] Daniel S. Berger, Ramesh K. Sitaraman, and Mor Harchol-Balter. 2017. AdaptSize: Orchestrating the Hot Object Memory Cache in a Content Delivery Network. In *USENIX NSDI*.

[6] Browser Use. 2024. BrowserUse: Open-Source Browser Automation for AI Agents. https://github.com/browser-use/browser-use.

[7] Michael Butkiewicz, Harsha V. Madhyastha, and Vyas Sekar. 2011. Understanding Website Complexity: Measurements, Metrics, and Implications. In *Proceedings of the ACM Internet Measurement Conference (IMC)*.

[8] Cloudflare. 2024. Bot Traffic Report 2024. https://radar.cloudflare.com/reports/bot-traffic. Accessed March 2026.

[9] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *NeurIPS Datasets and Benchmarks Track*.

[10] Gil Einziger, Roy Friedman, and Ben Manes. 2017. TinyLFU: A Highly Efficient Cache Admission Policy. *ACM Transactions on Storage* 13, 4 (2017).

[11] Phillipa Gill, Martin Arlitt, Zongpeng Li, and Anirban Mahanti. 2007. Youtube Traffic Characterization: A View from the Edge. In *Proceedings of the ACM Internet Measurement Conference (IMC)*.

[12] Google. 2022. Robots Exclusion Protocol. RFC 9309, IETF.

[13] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. 2024. WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models. In *Proceedings of the International Conference on Machine Learning (ICML)*.

[14] HTTP Archive. 2025. State of the Web. https://httparchive.org/. Accessed March 2026.

[15] Sunghwan Ihm and Vivek S. Pai. 2011. Towards Understanding Modern Web Traffic. In *Proceedings of the ACM Internet Measurement Conference (IMC)*.

[16] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? (2024).

[17] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. 2024. VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[18] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2024. AgentBench: Evaluating LLMs as Agents. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

[19] Xing Han Lù, Zdeněk Kasner, and Siva Reddy. 2024. WebLINX: Real-World Website Navigation with Multi-Turn Dialogue. (2024).

[20] Arunesh Mathur, Gunes Acar, Michael Friedman, Elena Lucherini, Jonathan Mayer, Marshini Chetty, and Arvind Narayanan. 2019. Dark Patterns at Scale: Findings from a Crawl of 11K Shopping Websites. In *Proceedings of the ACM Conference on Computer-Supported Cooperative Work and Social Computing (CSCW)*.

[21] Nimrod Megiddo and Dharmendra S. Modha. 2003. ARC: A Self-Tuning, Low Overhead Replacement Cache. In *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*.

[22] Microsoft. 2024. Playwright: Fast and Reliable End-to-End Testing. https://playwright.dev/.

[23] David Naylor, Alessandro Finamore, Ilias Leontiadis, Yan Grunenberger, Marco Mellia, Maurizio Munafò, Konstantina Papagiannaki, and Peter Steenkiste. 2014. The Cost of the "S" in HTTPS. In *Proceedings of the ACM Conference on Emerging Networking Experiments and Technologies (CoNEXT)*.

[24] Yanchen Peng, Zhiyuan Liang, and Srinivasan Seshan. 2024. Characterizing the Emerging AI-Generated Web Traffic. *arXiv preprint arXiv:2407.09175* (2024).

[25] TollBit. 2024. TollBit: Monetize AI Access to Your Content. https://tollbit.com/.

[26] Carl Waldspurger, Trausti Saemundsson, Irfan Ahmad, and Nohhyun Park. 2017. Cache Modeling and Optimization using Miniature Simulations. In *Proceedings of the USENIX Annual Technical Conference (ATC)*.

[27] x402 Foundation. 2025. x402: HTTP Payment Protocol. Coinbase and Cloudflare.

[28] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming

Xiong, Victor Zhong, and Tao Yu. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[29] Juncheng Yang, Ziyue Qiu, Yazhuo Zhang, Yao Yue, and Rashmi Vinayak. 2023. GL-Cache: Group-Level Learning for Efficient and High-Performance Caching. In *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*.

[30] Juncheng Yang, Yazhuo Zhang, Ziyue Qiu, Yao Yue, and Rashmi Vinayak. 2023. FIFO Can Be Better than LRU: The Power of Lazy Promotion and Quick Demotion. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*.

[31] Yuhan Zhang, Juncheng Cai, Ada Wildani, and Aleksandar Klimovic. 2025. Rethinking Web Cache Design for the AI Era. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*.

[32] Yazhuo Zhang, Juncheng Yang, Yao Yue, Ymir Vigfusson, and K. V. Rashmi. 2024. SIEVE is Simpler than LRU: an Efficient Turn-Key Eviction Algorithm for Web Caches. In *USENIX NSDI*.

[33] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *Proceedings of the International Conference on Learning Representations (ICLR)*.